

PPO 与 SAC 实验报告

南京大学

姓名： 施静宇

日期： 2026/8/16

实验概述

本次实验围绕两类经典深度强化学习算法展开：PPO (Proximal Policy Optimization) 与 SAC (Soft Actor-Critic)。PPO 是一种 on-policy 策略优化方法，通过 clipped surrogate objective 控制单次策略更新幅度；SAC 是一种 off-policy 最大熵强化学习方法，通过在优化累积回报的同时引入策略熵，兼顾性能优化与探索能力。

本报告按照任务要求分别完成 PPO 与 SAC 的核心代码实现、MuJoCo 环境实验及相关消融分析。PPO 部分重点考察若干 implementation details 对训练性能与稳定性的影响；SAC 部分重点研究固定 temperature 参数对学习过程的影响。实验曲线均优先以 `interaction_step` 为横轴，以避免不同训练配置下仅比较 epoch 数造成歧义。

PPO 与 SAC 原论文分别见：[Proximal Policy Optimization Algorithms](#) 与 [Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor](#)。

1 PPO 实验报告

1.1 代码任务

本次 PPO 代码补全位于 `algorithm/ppo.py`，核心包括两个函数：

- `_update_with_minibatch()`：负责单个 minibatch 上的 actor/critic 更新；
- `_update_policy()`：负责整理一次采样得到的数据，并完成多轮 minibatch 更新。

任务原題：请补全文件 `/algorithm/ppo.py` 中的 TODO 部分，包括 `_update_with_minibatch` 和 `_update_policy` 两个核心函数。完成后，请结合代码回答下述问题：

1. PPO clipped surrogate objective 的具体实现公式是什么？请解释 clip 操作如何限制策略更新幅度，以及它为什么可以缓解策略更新过大的问题。

答：

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

以一个 timestep $A > 0$ 时为例，此时 ratio 在奖励下会增大，为了防止 ratio 值过大，引入 clip 函数，当 ratio 的值超过 $1 + \epsilon$ 我们就认为新旧策略相差过大了，此时引发 clip 使得 ratio 仍然在我们控制的区间内，从而控制了策略更新幅度；

正如上面描述的那样，当 A 值过大或过小即将对策略产生较为激进奖励信号时，clip 机制就会触发，使得 ratio 被控制在一个稳定的平台上，也就缓解了更新过大的问题。

同时，为了提高 sample efficiency，PPO 在一批数据上会更新多次，但是随着更新次数的增多，clip 也会经常被触发，导致后续 actor 学不动。

2. entropy loss 在 PPO 中起什么作用？当 `entropy_coef` 过大或过小时，可能分别带来什么影响？

答：起到在 objective 的学习过程中奖励 exploration 的作用；

过大时，exploration 影响大于 exploitation，策略收敛不到最优，最终学习的效果差；

过小时，exploration 不够，对其他动作探索不足，也可能导致最终学习的策略效果不好。

3. 在 `_update_policy` 中，请解释为什么 trajectory 中需要使用 mask 去除 padding 的无效 transition 而 rollout 中不需要。

答：traj 模式是以 trajectory 为单位收集数据，不同 trajectory 之间长度也不尽相同，存储时通过 padding 将同一组 trajectory 补齐为相同长度，并通过 mask 记录一条 trajectory 从什么时候开始的数据是 padding 的，通过 critic 网络后，padding 处的数据可能变为非零值，此时就需要与 mask 做哈德玛积来保证数据真是性；

rollout 模式是按固定 timestep 来收集数据，收集到的数据都是真实的，故不需要 padding,mask 机制。

4. advantage normalization 的实现位置在哪里？它会改变策略梯度的方向吗？为什么通常可以提升训练稳定性？

答：位置在从 buffer 取出 transition 数据并计算出 advantage 之后，在 minibatch update 之前；AN 的公式是： $\hat{A}_t^{\text{norm}} = \frac{\hat{A}_t - \mu_A}{\sigma_A + \epsilon}$ 。分母除以一个正的标准差不会改变策略更新的方向，分子减去一个与 action 无关的 baseline 不会引入偏差，同样不会改变方向；

AN 机制通过将一个小 batch 中的 advantage 数据统一到一个尺度内，可以有效防止某个极端值主导了策略更新，同时减小了不同 minibatch 之间梯度的波动，从而提升了训练的稳定性。

5. 请列出你最终实现的 PPO 总 loss，并说明 actor loss、critic loss、entropy loss 在总目标中的符号和权重。

答：`total_loss=actor_loss+self.value_coef*critic_loss-policy ENTR*self.entropy_coef`

`actor_loss` 由于已经将 surrogate objective 符号变号，且要增大 objective，即减小 actor loss，所以符号为正；由于要减小 critic loss，所以符号为正，权重是 `self.value_coef`；由于要增大 entropy 保证 exploration，所以符号为负，权重是 `self.entropy_coef`。

1.2 PPO 关键公式

任务：请在本节写出 PPO 的优化目标，并解释每一项的含义：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

其中，策略概率比通常记为

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$

答： $r_t(\theta)$ 表示当前策略相较于旧策略对 a_t 的偏好权重；

$\hat{A}_t$ 表示 advantage，即相较于 baseline $V(s_t)$ ，当前动作价值 $Q(a_t, s_t)$ 提升或降低了多少；

$\mathbb{E}_t = [r_t(\theta) \hat{A}_t]$ 是 surrogate objective 的原始等式。

1. 当 $\hat{A}_t > 0$ 时，clip 机制如何限制提升动作概率的幅度？

答： 当 $\hat{A}_t > 0$ 时，如果奖励太大导致 ratio 值超过 $1 + \epsilon$ ，触发 clip 机制，此时 $r_t(\theta) \hat{A}_t > (1 + \epsilon) \hat{A}_t$ ，所以 $L^{\text{CLIP}}(\theta) = \mathbb{E}_t [(1 + \epsilon) \hat{A}_t]$ ，这样就限制了过大地提高动作概率的幅度。

2. 当 $\hat{A}_t < 0$ 时，clip 机制如何限制降低动作概率的幅度？

答： 当 $\hat{A}_t < 0$ 时，如果奖励太小导致 ratio 值小于 $1 - \epsilon$ ，触发 clip 机制，此时 $r_t(\theta) \hat{A}_t > (1 - \epsilon) \hat{A}_t$ ，所以 $L^{\text{CLIP}}(\theta) = \mathbb{E}_t [(1 - \epsilon) \hat{A}_t]$ ，这样就限制了过大地降低动作概率的幅度。

3. PPO 与 TRPO 的联系是什么？二者分别如何控制新旧策略之间的差异？

答： 两者都是通过提高 surrogate objective 来保证策略更新是向好的方向进行的，只不过是控制新旧策略距离的方法不同：

TRPO 通过共轭梯度法求出 actor 参数更新的方向，再通过限制新旧策略之间的 KL 距离来求出最大步长，再用 linear search 方法求出满足约束条件下新 actor；PPO 则是通过 clip 机制对 surrogate objective 进行改造，保证了新旧策略的 ratio 在一个可控范围内，防止出现大幅度的策略更新。

另外，指的是注意的是，TRPO 在一个 epoch 内只更新了一次 critic 和 actor，而 PPO 在一个 epoch 内进行了多次策略更新，并且仍表现了较好的学习效果和稳定性，同时也降低了样本复杂度，说明在本实验的情况下，PPO 是更先进的策略梯度更新算法。

1.3 PPO 实验设置

任务： 请从以下 MuJoCo 环境中至少选择三个进行测试：**Humanoid-v5**, **Ant-v5**, **HalfCheetah-v5**, **Hopper-v5**, **Walker2d-v5**。

1. 给出每个环境的基础超参数设置。

2. 每组实验运行 1-3 个随机种子，并报告平均回报曲线。

本实验选择 **HalfCheetah-v5**、**Hopper-v5**、**Walker2d-v5** 三个连续控制环境。正式实验均使用 `seed=0`。由于实验预算限制，本报告不将单随机种子结果表述为多随机种子统计均值，而仅分析 `seed=0` 下的实际训练现象。

表 1: PPO 基础实验超参数记录表

环境	Seed	总 epoch	每 epoch 交互数	Minibatch	环境相关关键设置
HalfCheetah-v5	0	3000	128	256	obs norm: true; rescale: true; return scaling: false; initialize: true
Hopper-v5	0	1500	256	256	obs norm: true; rescale: false; return scaling: true; lr decay: true
Walker2d-v5	0	1500	256	256	obs norm: true; rescale: false; return scaling: true; lr decay: true

三组 baseline 均使用 `update_epochs=5`。共同设置还包括：

- `eps_clip=0.2`, `actor_lr=3e-4`, `critic_lr=3e-4`;
- `use_value_clip=true`, `value_clip=0.2`;
- `use_grad_clip=true`, `max_grad_norm=0.5`;
- `advan_norm=false`, `collect_traj=false`, `buffer_name=ReplayBuffer`。

evaluation 曲线统一以 `interaction_step` 为横轴。

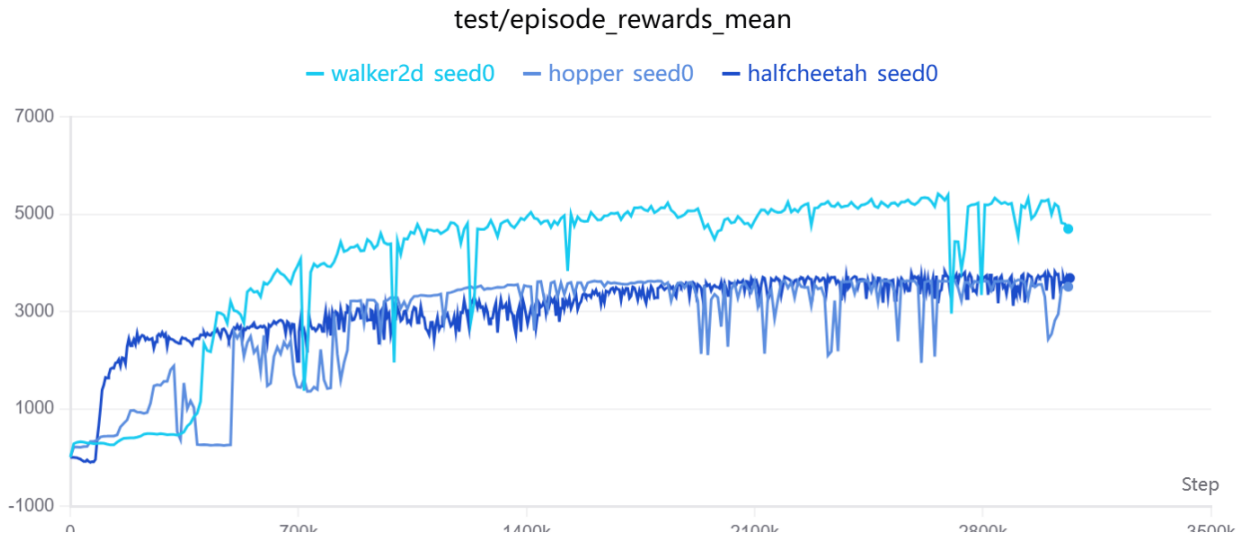


图 1: PPO 在三个 MuJoCo 环境上的 evaluation mean reward 曲线

如图 1 所示，PPO 在 HalfCheetah-v5、Hopper-v5 和 Walker2d-v5 三个环境中均能够实现明显的策略改进，但三个环境表现出了不同的学习曲线。HalfCheetah-v5 在训练前期的回报提升最快，在较少的环境交互后即可达到较高水平，随后进入较为平缓的持续提升阶段，整体训练过程相对稳定。Walker2d-v5 和 Hopper-v5 在训练初期提升较慢，但随后回报迅速增长，并在中后期进入相对稳定的平台阶段，但后期训练中仍有较为明显的性能波动。

1.4 PPO 37 Implementation Details 探索

任务：请参考 PPO 37 implementation details，从下述实现细节中至少选择 3 个进行消融实验，例如：obs_norm、return_scaling、advan_norm、rescale、lr_decay、use_value_clip、use_grad_clip、log_clip_stabilize 等。

建议每次只改变一个实现细节，其余超参数保持不变。请按照如下结构撰写每个细节的实验与分析：

1. 该实现细节在代码或配置文件中的对应参数是什么？默认值是什么？
2. 它从算法角度解决了什么问题？例如数值稳定、梯度尺度、状态分布漂移、动作范围匹配、回报尺度变化等。
3. 实验对照组如何设置？请明确写出 baseline 与 ablation 的区别。
4. 实验曲线有何差异？请从收敛速度、最终性能、曲线方差、是否出现训练崩溃等角度分析。
5. 你认为产生上述现象的原因是什么？请结合 PPO 的更新公式或环境特点进行解释。

本实验选择 **value clipping**、**observation normalization**、**gradient clipping** 三项实现细节进行消融。三组消融均在 HalfCheetah-v5、seed=0 下进行，并复用同一 HalfCheetah baseline；每次只改变一个配置项。

表 2: PPO implementation details 消融实验记录表

环境	实现细节	Baseline	Ablation
HalfCheetah-v5	use_value_clip	true	false
HalfCheetah-v5	obs_norm	true	false
HalfCheetah-v5	use_grad_clip	true	false

1.4.1 实现细节一：Value Clipping

参数与对照设置 对应配置项为 algorithm.use_value_clip。Baseline 中设置为 true，消融组仅将其修改为 false；其余 HalfCheetah-v5 训练参数保持不变。

题目 1: 该实现细节在代码或配置文件中的对应参数是什么？默认值是什么？

答: 对应参数为 `algorithm.use_value_clip`, 配置文件默认值为 `true`; value clipping 的阈值由 `algorithm.value_clip=0.2` 给出。

题目 2: 它从算法角度解决了什么问题？例如数值稳定、梯度尺度、状态分布漂移、动作范围匹配、回报尺度变化等。

答:

$$V_{\text{clip}}(s) = V_{\text{old}}(s) + \text{clip}(V_{\text{new}}(s) - V_{\text{old}}(s), -\epsilon_v, \epsilon_v),$$

$$L_V = \frac{1}{2} \mathbb{E} [\max((V_{\text{new}}(s) - V_{\text{target}}(s))^2, (V_{\text{clip}}(s) - V_{\text{target}}(s))^2)].$$

解决了新旧 critic 差异过大更新过快的问题，否则理论上可能面临以下几个问题：

1. 由于同一批 minibatch 上的 target value 并不是一个精确确定的真实值，所以过度追随 target 可能会导致数据上的过拟合，同时不同 minibatch 之间可能由于噪声等原因有较大的差异，此时如果采用较为激进的更新方式可能会导致训练上较为不稳定；

2. critic 的学习会通过 advantage 的计算传回 actor, 若是由于上面所说导致了 critic 的不稳定，可能也会导致 actor 的不稳定，最终导致整体训练的波动；

于是通过和 surrogate objective clip 类似的设计，使得 critic 更新限制在了一个稳定的范围内，提高了更新数值的稳定性，对其他几个方面并无明显影响。

题目 3: 实验对照组如何设置？请明确写出 baseline 与 ablation 的区别。

答: baseline 使用 `use_value_clip=true`; ablation 使用 `use_value_clip=false`, 两组均采用 HalfCheetah-v5、`seed=0`, 并保持其他训练设置一致。

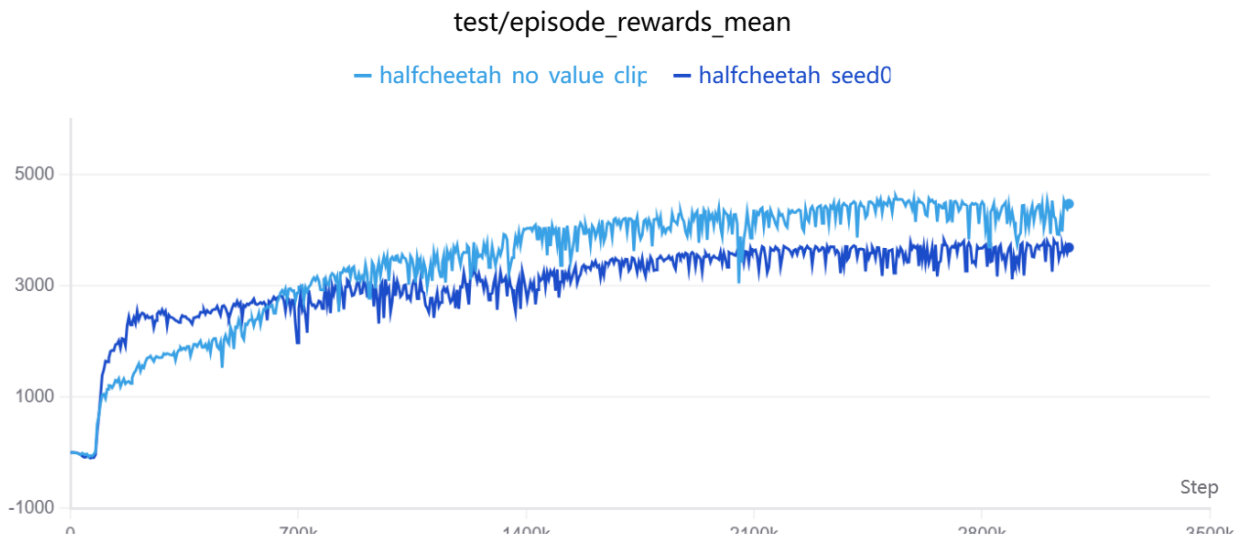


图 2: HalfCheetah-v5 上 value clipping 消融实验

题目 4: 实验曲线有何差异？请从收敛速度、最终性能、曲线方差、是否出现训练崩溃等角度分析。

答: 本组实验配置条件下，前期 baseline 组训练较快，效果较好，但虽然前期 ablation

组训练得较慢，但是 700k 之后，ablation 组训练回报开始超过 baseline，700k 之后两组均缓慢增长但是 ablation 组始终维持在比 baseline 回报高的水平。

收敛速度上 baseline 组收敛更快；最终性能上 ablation 进入平台期后回报稳定高于 baseline；波动性上两者接近，方差相差不大。

题目 5：你认为产生上述现象的原因是什么？请结合 PPO 的更新公式或环境特点进行解释。

答：可以猜测的原因是 value clip 机制主要是为了保证训练数值稳定性，保守更新。但当 critic 更新本来就比较稳定时，这种保守机制可能反而拖累了 critic 的更新，进而导致了训练回报的缓慢增长；通过这个 ablation 实验可以发现 value clip 会面临稳定性和 critic 更新速率的 trade-off。既然 value clip 可能会带来过于保守的负面的效果，那 PPO 算法的核心：surrogate objective 的 clip 机制会不会也带来这种过于保守的后果呢？理论上应该是会的，但是相比于激进更新带来最后训练的崩溃，我们更愿意取牺牲一点更新的速率。

1.4.2 实现细节二：Observation Normalization

对照实验设置 对应配置项为 `env.obs_norm`。Baseline 中为 `true`，消融组仅修改为 `false`，其余 HalfCheetah-v5 配置保持不变。

任务：

1. 对照实验设置：
2. 实验结果：
3. 原因分析：

对照实验设置：baseline 使用 `env.obs_norm=true`；ablation 使用 `env.obs_norm=false`。两组均采用 HalfCheetah-v5、`seed=0`，其他配置保持一致。

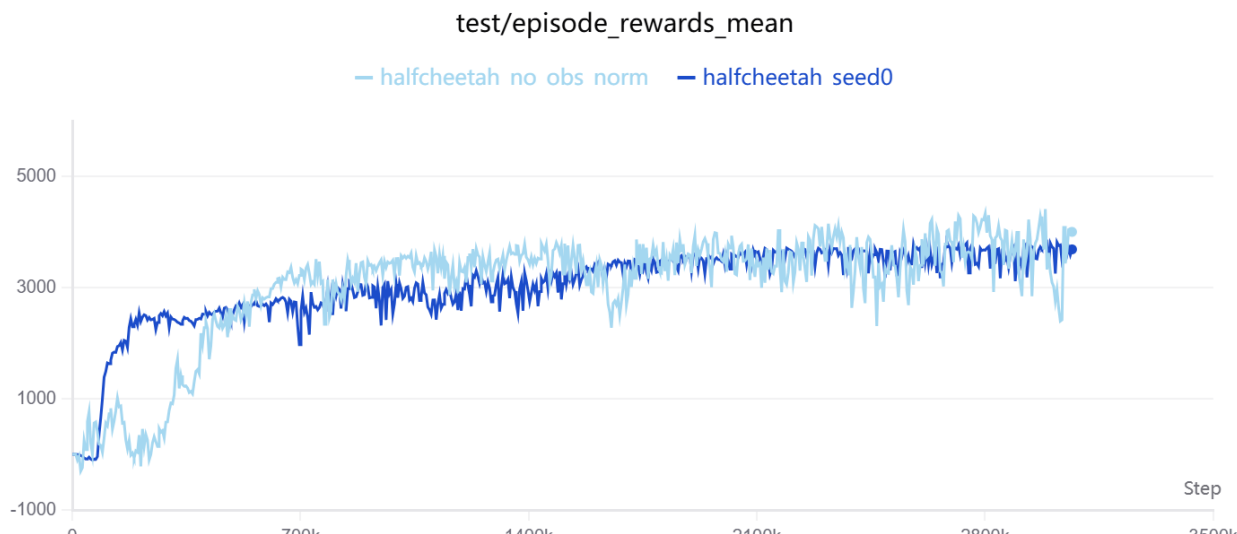


图 3: HalfCheetah-v5 上 observation normalization 消融实验

实验结果

答：baseline 收敛速度更快，较早的达到平台期，并且较早进入平台期，整体波动较小，进入平台期后较为稳定；ablation 组早期波动剧烈，收敛速度明显慢于 baseline，平台期仍有较为明显的波动，整体稳定性上明显弱于 baseline；最终 reward 两组在相近的水平。

原因分析

答：observation normalization 通过归一化，将不同维度的状态信息维持在相近的尺度上。由于状态有多重维度的信息，不同维度信息数值尺度可能相差很大，在神经网络中，数值大的信息可能对神经元激活和梯度尺度产生不成比例的影响，导致数值尺度的信息利用不充分，网络优化变差。所以，通过归一化可以更合理的利用状态信息优化神经网络，也减小了不同维度数值尺度相差过大导致的训练波动，最终使得 baseline 的训练较为稳定。

1.4.3 实现细节三：Gradient Clipping

对照实验设置 对应配置项为 `algorithm.use_grad_clip`。Baseline 中为 `true`，消融组仅修改为 `false`；启用时最大梯度范数由 `max_grad_norm=0.5` 给出。

任务：

1. 对照实验设置：
2. 实验结果：
3. 原因分析：

对照实验设置：baseline 使用 `use_grad_clip=true`；ablation 使用 `use_grad_clip=false`。两组均采用 HalfCheetah-v5、`seed=0`，其他配置保持一致。

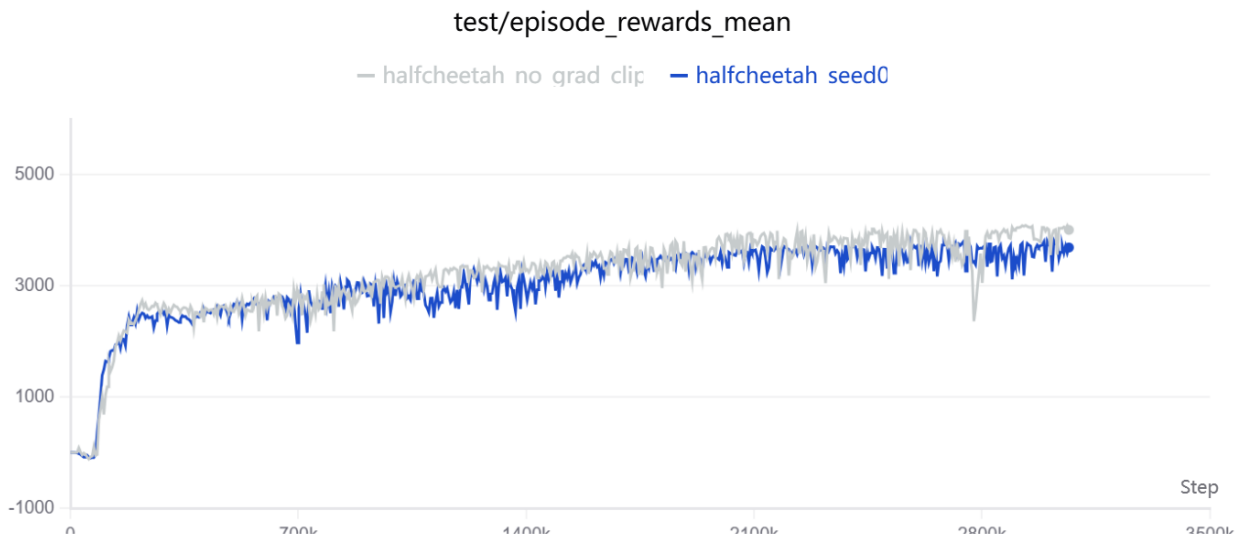


图 4: HalfCheetah-v5 上 gradient clipping 消融实验

实验结果

答：如图，baseline 与 ablation 高度重合，两者趋势相近，在训练曲线上都是早期学习速率较快，在 1400k 左右进入平台期，后期训练伴随着一定的波动，从曲线上看，该配置信息下 ablation 未对训练过程产生明显影响。

原因分析

答：gradient clipping 机制是：当梯度尺度超过某个阈值时，就将梯度向量等比例缩放，使得梯度的 L2 范数恰好等于该阈值，于是将梯度尺度控制在了一个较为稳定的区间上。

本组 ablation 实验没有产生明显差异可能是因为 PPO 的 surrogate objective 等优化机制在一定程度上已经控制了梯度尺度的大小，并没有出现太多 gradient 尺度过大的情况，使得 gradient clipping 机制并没有发挥太大的作用。

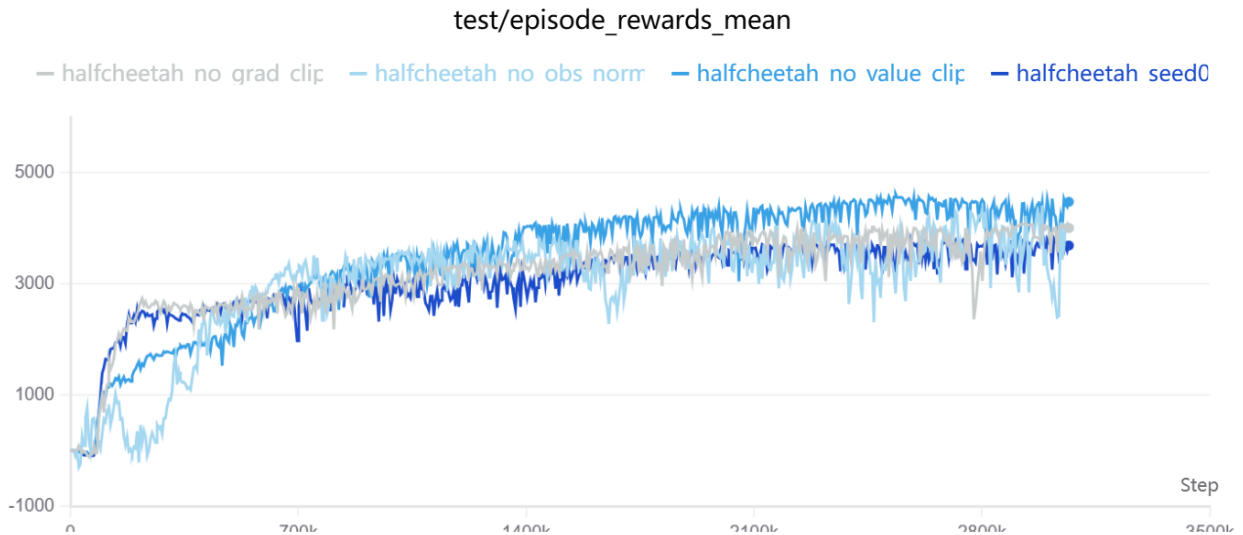


图 5: HalfCheetah-v5 上三组消融实验对比

1.5 PPO 实验结果分析

1. 在你选择的三个环境中，PPO 的训练难度是否一致？请结合 reward 曲线和环境本身的状态维度、动作维度、终止条件进行分析。

答：从曲线波动性上来看，halfcheetah 曲线波动性最小，训练最稳定，walker 和 hopper 波动性都较大，训练稳定性上明显逊于 halfcheetah。

2. 哪些 implementation details 对 PPO 的性能影响最大？这些影响是否在所有环境中一致？

答：在 halfcheetah 环境中消融实验对原本实验效果产生较明显负面效果的是 observation normalization；value clipping 的 ablation 反而优化了实验结果；gradient clipping 的 ablation 对实验影响不大。

由于计算资源的限制，本次 PPO 的三组消融实验仅在 halfcheetah 环境中进行。

3. PPO 的训练是否出现了高方差、性能回退或训练崩溃？如果有，请结合 ratio、advantage、entropy、value loss 等指标分析可能原因。

答：本次实验的三组 baseline 和三组 ablation 实验训练曲线都较为正常，并没有出现类似 DQN 学习率 sensitivity 实验中出现的训练崩溃和性能回退的现象。方差差异排除不同环境本身就可能带有的差异这个因素之外，在 observation normalization 消融实验中方差明显变高，可能原因上面已经分析过了。

总体来说，PPO 的 clipping 机制还是很大程度上保证了训练的稳定性，避免了策略梯度方法中常常因为学习过于剧烈导致的训练崩溃现象。

2 SAC 实验报告

2.1 代码任务

本次 SAC 代码补全位于 `algorithm/sac.py`。一次参数更新包括 replay buffer 采样、soft Bellman target 构造、双 Q critic 更新、actor 更新、temperature 更新以及 target network 更新。

任务：请补全文件 `/algorithm/sac.py` 中的 TODO 部分，实现 SAC 的一次参数更新，包括 replay buffer 采样、soft Bellman target、双 Q critic 更新、actor 更新、temperature 更新和 target network 更新。完成后，请结合代码回答下述问题：

1. soft Bellman backup 的目标值如何计算？请写出 target Q 的公式，并解释其中 entropy 项的作用。

答：soft Bellman backup 的目标值指的是 critic 的 soft target: $y_t = r_t + \gamma(1 - d_t) [\min_i Q_{\phi_i}(s_{t+1}, a_{t+1}) - \alpha \log \pi_{\theta}(a_{t+1} | s_{t+1})]$ ，其中 entropy bonus 的单样本估计是 $-\alpha \log \pi_{\theta}(a_{t+1} | s_{t+1})$ 。它用来描述策略的分散程度，entropy 值越大，策略的动作分散程度越高，动作的随机性越强，越有利于维持 exploration，反之，策略的动作概率分布越集中，确定性越强。将 entropy 项引入 critic target，形成 soft Bellman backup，使得 Q-function 在评价 (s,a) 时会同时考虑未来回报和未来 policy 的 entropy。

2. 为什么 SAC 通常使用 double critic？target 中为什么取两个 Q 值的最小值？

答：设计两个独立不共享参数的 critic network，是为了防止因为 critic 错误地高估某个动作，进而导致 actor 对该动作的错误选择，从而进入恶性循环，加大误差。取最小值正是为了防止上述恶性循环的保守策略，牺牲可能更优的策略选择换取训练的稳定。

3. actor 更新时为什么需要重新从当前策略采样动作，而不是直接使用 replay buffer 中保存的旧动作？

答： SAC 是 off-policy 算法，因此 actor 更新时状态 s_t 可以直接从 replay buffer D 中采样，而不要求这些状态由当前策略产生。根据 SAC 的 actor objective,

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim D, \epsilon_t \sim \mathcal{N}} [\log \pi_\phi(a_t | s_t) - Q_\theta(s_t, a_t)], \quad a_t = f_\phi(\epsilon_t; s_t),$$

其中动作 a_t 必须由当前策略 $\pi_\phi(\cdot | s_t)$ 重新采样。replay buffer 中保存的旧动作由历史 behavior policy 产生，通常不再服从当前策略的动作分布，因此若直接使用旧动作，就不能正确估计当前 actor objective 中关于 $a_t \sim \pi_\phi$ 的期望。重新采样动作只需要对当前 actor 进行一次前向计算，并不需要重新与环境交互。

$$\mathcal{T}^\pi Q(s_t, \mathbf{a}_t) \triangleq r(s_t, \mathbf{a}_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V(s_{t+1})],$$

其中

$$V(s_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi} [Q(s_t, \mathbf{a}_t) - \log \pi(\mathbf{a}_t | s_t)]$$

同样说明了当 state 确定时，仍需要从当前策略中采样得到 action 交互数据。

4. 如果 learn_temp=true, temperature 的优化目标是什么？

答： entropy_target=self.target_entropy, 通过 temperature 的梯度更新使得 policy entropy 维持在self.target_entropy 的水平。

2.2 SAC 策略优化目标证明

任务： 请证明 SAC 中策略 π 的优化目标为什么可以写成当前代码中的形式：

$$J_\pi(\theta) = \mathbb{E}_{s_t \sim D, \epsilon_t \sim \mathcal{N}} [\alpha \log \pi_\theta(f_\theta(\epsilon_t; s_t) | s_t) - Q_\phi(s_t, f_\theta(\epsilon_t; s_t))].$$

答： 假设此时 $Q^{\text{old}}(s, a)$ 已经评估好了

固定状态 s ，欲选择一个动作分布最大化 $\mathbb{E}_{a \sim \pi} [Q(s, a)] + \mathcal{H}(\pi)$

离散情况下就是 $\max [\sum_a \pi(a|s) Q(s, a) - \sum_a \pi(a|s) \log \pi(a|s)]$

其中 $\sum_a \pi(a|s) = 1$

由 Lagrange 乘数法：

$$\mathcal{L}(\pi_a, \lambda) = \sum_a \pi_a Q_a - \sum_a \pi_a \log \pi_a + \lambda \left(\sum_a \pi_a - 1 \right)$$

$$\frac{\partial \mathcal{L}}{\partial \pi_a} = Q_a - \log \pi_a - 1 + \lambda = 0$$

$$\Rightarrow \log \pi_a = Q_a + \lambda - 1$$

$$\Rightarrow \pi_a \propto \exp(Q_a)$$

$$\text{归一化后 } \pi^*(a|s) = \frac{\exp(Q(s, a))}{Z(s)}, \quad Z(s) = \sum_a \exp(Q(s, a))$$

但 Gaussian policies 未必能精确表示 π^*
于是算法中

$$\begin{aligned} \pi_{\text{new}} &= \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot|s) \parallel \frac{\exp(Q^{\text{old}}(s, \cdot))}{Z^{\text{old}}(s)} \right) \\ &= \arg \min_{\pi' \in \Pi} \mathbb{E}_{a \sim \pi'} [\log \pi'(a|s_t) - Q^{\text{old}}(s, a) + \log Z^{\text{old}}(s_t)] \end{aligned}$$

考虑到 $Z(s)$ 是常数, 再引入 temperature α
令 $a_t = f_\theta(\varepsilon_t; s_t)$, $\varepsilon_t \sim \mathcal{N}$
所以 objective 是最大化

$$J_\pi(\theta) = \mathbb{E}_{s_t \sim D, \varepsilon_t \sim \mathcal{N}} [\alpha \log \pi_\theta(f_\theta(\varepsilon_t; s_t) | s_t) - Q_\phi(s_t, f_\theta(\varepsilon_t; s_t))]$$

2.3 SAC 实验设置

任务原题: 请从以下 MuJoCo 环境中至少选择三个进行测试: **Humanoid-v5**, **Ant-v5**, **HalfCheetah-v5**, **Hopper-v5**, **Walker2d-v5**。

1. 给出每个环境的基础超参数设置, 例如 `total_epoch`、`buffer_size`、`batch_size`、`actor_lr`、`critic_lr`、`init_temp`、`learn_temp`、`target_update_tau` 等。
2. 每组实验建议运行 1-3 个随机种子, 并报告平均值和方差。

本实验选择 **HalfCheetah-v5**、**Hopper-v5**、**Walker2d-v5** 三个环境。三组 baseline 均采用固定 temperature, 设置 `learn_temp=false`、 $\alpha = 0.2$ 、`seed=0`。由于本次正式实验只运行单随机种子, 本报告不将结果表述为多随机种子的均值与方差。

表 3: SAC 基础实验超参数记录表

环境	Seed	总 epoch	初始 α	学习温度	Batch	Buffer
HalfCheetah-v5	0	1,500,000	0.2	false	256	1,000,000
Hopper-v5	0	1,000,000	0.2	false	256	1,000,000
Walker2d-v5	0	1,000,000	0.2	false	256	1,000,000

共同超参数包括:

- actor_lr=3e-4, critic_lr=3e-4, start_train_step=5000;
- interact_per_epoch=1, update_step_per_epoch=1;
- target_update_method=soft, target_update_interval=1, target_update_tau=0.005;
- double_critic=true, rescale=true。

跨环境 baseline 比较时统一考察前 1.0×10^6 个 interaction steps。

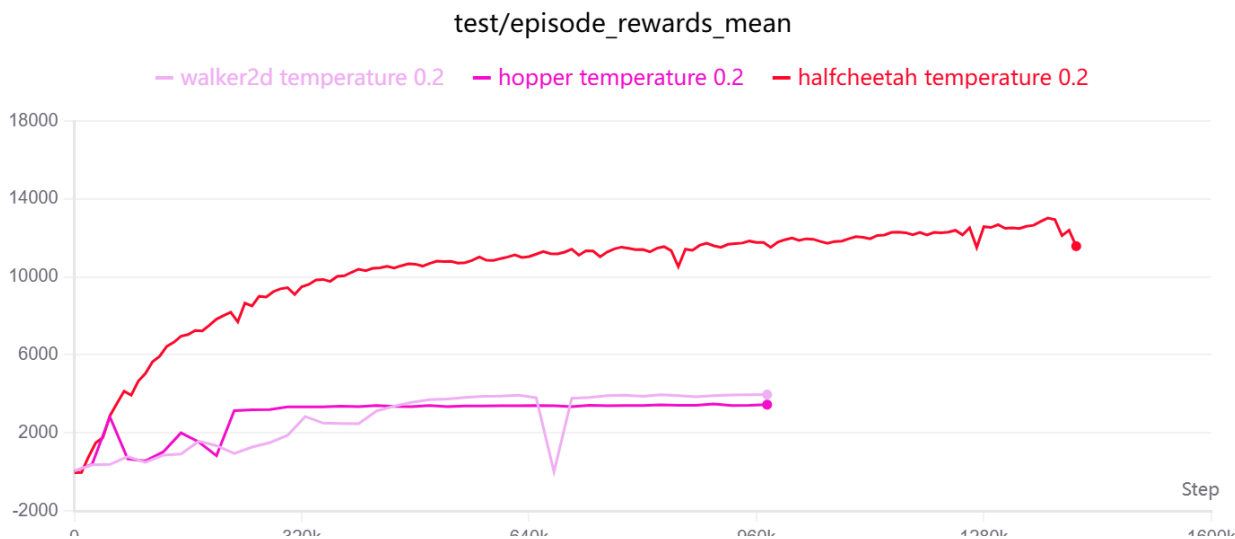


图 6: SAC 在三个 MuJoCo 环境上的 baseline 训练曲线

如图, halfcheetah 环境整体训练较为稳定, 波动整体较小; hopper 环境下, 在 300k 之前训练波动性很大, 300k 之后基本维持在水平线; walker 环境 700k 之前训练波动性很大, 之后同样维持在一个水平线; 总的来说 hopper, walker 环境下训练效果不如 halfcheetah。

2.4 固定 Temperature 的影响

任务: 本节要求设置 `learn_temp=false`, 并尝试不同固定温度 α 的取值, 例如 $\alpha \in \{0.05, 0.1, 0.2, 0.5, 1.0\}$ 。请分析不同 temperature 对实验性能的影响。

1. 当 α 较小或较大会对探索性能和训练稳定性产生什么影响?
2. 不同环境对 temperature 的敏感程度是否一致? 请结合环境动作维度、奖励尺度、稀疏程度和局部最优等一些角度进行分析。
3. 固定 temperature 与自动调节 temperature 相比, 各自有什么优缺点?

题目给出的 $\{0.05, 0.1, 0.2, 0.5, 1.0\}$ 为候选取值示例。结合本次实验预算, temperature sensitivity 实验选择 HalfCheetah-v5 与 Hopper-v5 两个环境, 并在每个环境中考

察 $\alpha \in \{0.05, 0.2, 0.5\}$ 。其中 $\alpha = 0.2$ 直接复用对应环境 baseline，额外运行 $\alpha = 0.05$ 与 $\alpha = 0.5$ 两组实验。三条曲线统一比较前 7.5×10^5 个 interaction steps。

2.4.1 HalfCheetah-v5 Temperature Sensitivity

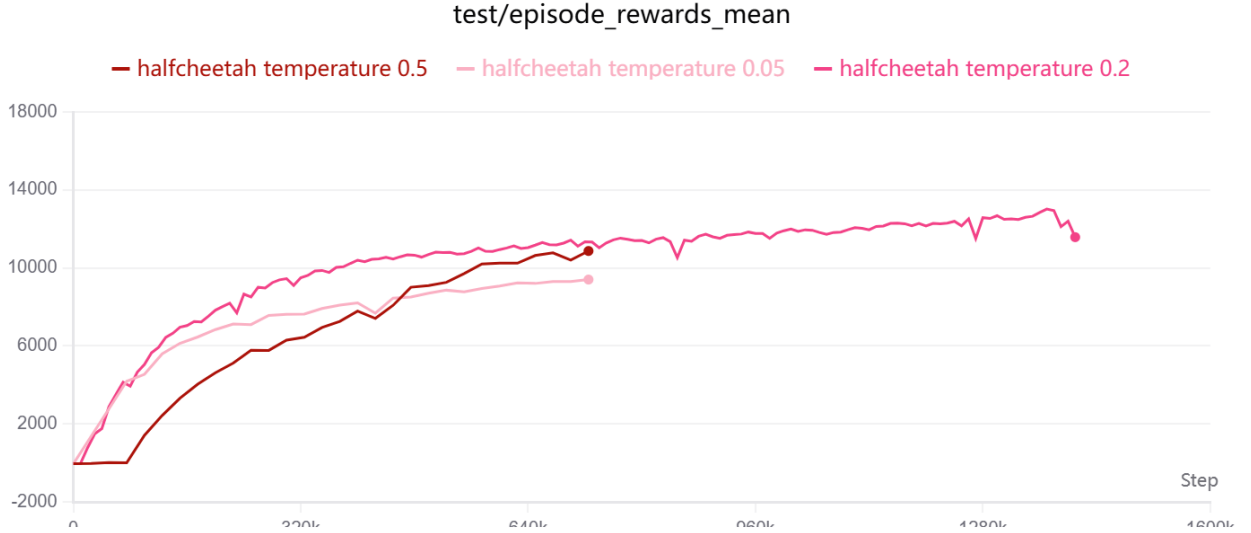


图 7: HalfCheetah-v5 在不同固定 temperature 下的 reward 曲线

实验结果：如图，在三组实验共同的 0–750k interaction steps 区间内，不同固定 temperature 对 SAC 的训练过程产生了较明显的影响。取 $\alpha=0.2$ 时，evaluation reward 在大部分训练阶段均高于另外两组，表现出较快的学习速度和较好的中后期性能；取较小的 $\alpha=0.05$ 时，训练初期 reward 提升较快，但中后期增长逐渐放缓；取较大的 $\alpha=0.5$ 时，训练初期的学习速度明显较慢，但随着训练继续，其 reward 持续提高，并在中后期超过 $\alpha=0.05$ ，逐渐接近 $\alpha=0.2$ 。因此，在本实验的 HalfCheetah-v5、seed=0 配置下， $\alpha=0.2$ 表现出了较好的整体训练效果。

原因分析：SAC 中 temperature α 控制 entropy regularization 相对于 Q value 的权重。较小的 α 使策略更倾向于利用当前 critic 认为价值较高的动作，因而可能具有较快的早期性能提升，但较弱的策略随机性也可能限制后续探索；较大的 α 则使策略更加重视保持较高 entropy，增强探索能力，但同时降低了训练早期对已有高价值动作的利用程度，因此学习速度较慢。本实验中 $\alpha=0.5$ 前期落后但后期逐渐追赶，而 $\alpha=0.05$ 较早出现性能增长放缓，与上述 exploration–exploitation trade-off 相符。相比之下， $\alpha=0.2$ 在本实验中对探索和利用取得了较好的平衡。

2.4.2 Hopper-v5 Temperature Sensitivity

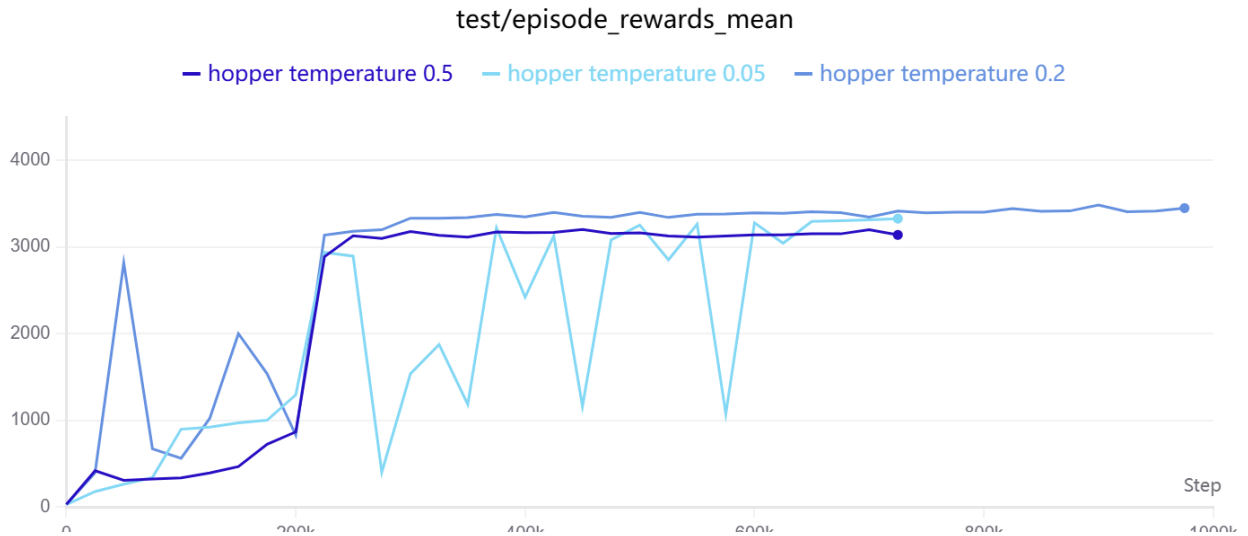


图 8: Hopper-v5 在不同固定 temperature 下的 reward 曲线

实验结果：如图 $\alpha=0.2$ 在训练早期存在一定波动，但约 300k steps 后迅速进入较高且稳定的性能平台，并在中后期保持三组中最好的整体性能。 $\alpha=0.5$ 的早期学习速度较慢，但达到较高 reward 后表现出较好的稳定性，最终性能仅略低于 $\alpha=0.2$ 。相比之下， $\alpha=0.05$ 虽然多次达到与另外两组相近的 reward 水平，但中后期出现多次明显的性能回退，训练稳定性显著较差。

原因分析：较小的 α 使策略更加倾向于利用当前 critic 估计的高价值动作，但降低了策略随机性，让 actor 更偏向于追逐 Q，从而使得策略更新变得更激进。在 Hopper-v5 这类对身体平衡和动作变化较为敏感的任务中，过低的策略随机性可能使 policy 更集中于某一特定动作模式，当 critic 估计产生误差时，更容易产生明显的性能退化。较大的 $\alpha=0.5$ 则保持了较强的 exploration，因此早期学习较慢，但训练后期表现更加稳定。本实验中 $\alpha=0.2$ 在学习速度、最终性能和训练稳定性之间取得了较好的平衡。

问题 1：当 α 较小或较大会对探索性能和训练稳定性产生什么影响？

答：正如上述两组实验分析的那样， α 较小时，策略更新更偏向于当前 critic 估计的高价值动作，这种局部的高确定性使得策略更新更容易加重 critic 的偶然误差，从而在一些敏感性高的环境中稳定性较差，当然， α 较小时，对未来策略的随机性关注自然会变小，探索不充分；

α 较大时，由于关注未来策略随机性的程度较大，在早期训练表现中回报会低于 α 较小的实验回报；但是由于策略探索较为充分，策略更新较为温和，两个环境中 α 较大的实验组表现得最为稳定，波动性最小；探索总归会在短期内牺牲对已知最优策略的利用程度以换取训练稳定性和策略的鲁棒性。

问题 2：不同环境对 temperature 的敏感程度是否一致？请结合环境动作维度、奖励尺度、稀疏程度和局部最优等一些角度进行分析。

答：halfcheetah 和 hopper 两个环境对 temperature 敏感性相差较大，虽然总体在训练回报上两者都符合上述分析，但 temperature 并未对 halfcheetah 的训练稳定性产生较大影响，而在 hopper 环境中高 temperature 明显使得数值稳定性得到提高，低 temperature 产生剧烈波动。

从动作维数上看，hopper 低于 halfcheetah，所以不能简单把环境敏感性归因于动作维数。从物理层面看，hopper 是单腿机器人，对局部最优动作的追求使得策略更新更剧烈，使得 critic 的误差更容易被放大，而动力学上 hopper 本身就更脆弱，对这种误差容忍度更低，使得该环境对 temperature 敏感性更高。

问题 3：固定 temperature 与自动调节 temperature 相比，各自有什么优缺点？

答：固定 temperature 的优点是实现简单、训练过程可控，并且便于直接分析不同 α 对策略探索程度和性能的影响，因此适合进行 sensitivity analysis。但固定 α 需要人工调节，其最佳取值往往依赖具体环境，而且无法根据训练阶段动态调整 exploration 与 exploitation 的平衡。

自动 temperature tuning 则通过给定 target entropy，并根据当前策略 entropy 动态调整 α ，使策略随机性维持在期望水平附近。其优点是能够减少对固定 temperature 的人工调参，并对不同环境和训练阶段具有更强的自适应能力；缺点是需要额外优化 temperature 参数，使训练过程更加复杂，同时其性能仍然受到 target entropy 选择的影响。

2.5 SAC 实验结果分析

任务：

1. 训练过程中 critic loss、actor loss、entropy、Q value 是否出现异常变化？这些指标如何帮助判断训练是否稳定？

actor loss: SAC 的 actor loss 由 entropy regularization 项与 Q value 项共同构成，因此其绝对值不能像监督学习 loss 一样用于直接判断策略性能。本实验中 actor loss 随训练过程平滑下降，未出现明显的剧烈振荡或突变，说明 actor 的优化过程总体较为稳定。其持续下降主要与策略逐渐选择更高 Q value 的动作有关，因此需要结合 evaluation reward 和 Q value 共同判断训练效果。

critic loss: critic loss 衡量当前 Q prediction 与 soft Bellman target 之间的误差。本实验中 critic loss 大部分时间维持在相对稳定的范围内，仅在少数训练阶段出现较明显的瞬时 spike，随后能够较快恢复。这说明训练过程中偶尔出现较大的 TD error，但并未形成持续性发散。结合 evaluation reward 持续提升以及 actor loss 的平稳变化，可以认为当前 SAC baseline 的整体训练过程较为稳定。



图 9: HalfCheetah-v5 baseline actor loss 曲线

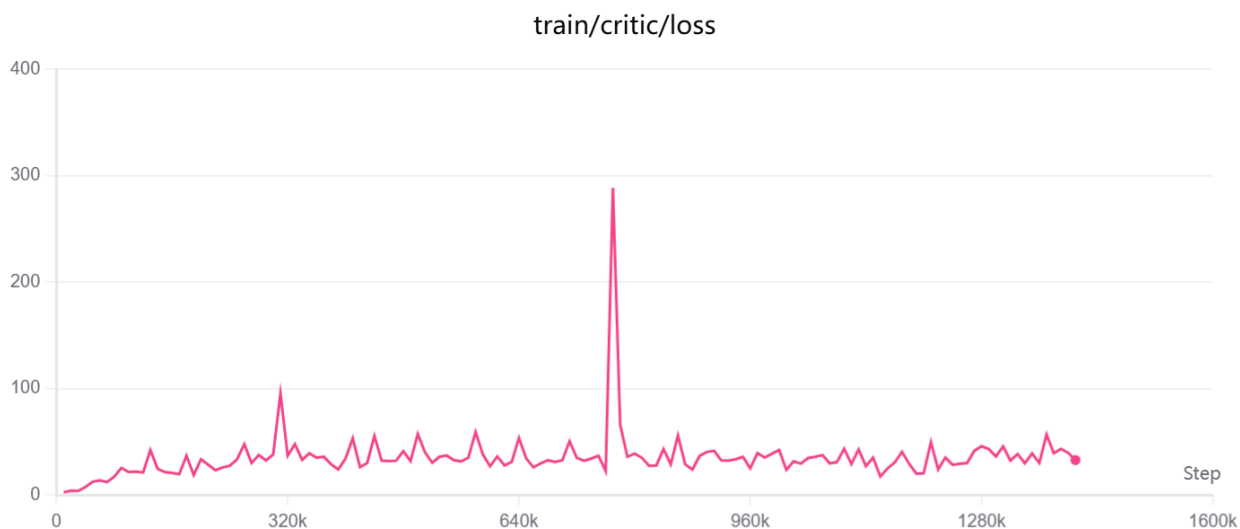


图 10: HalfCheetah-v5 baseline critic loss 曲线

3 PPO 与 SAC 对比分析

1. 从数据使用方式看，PPO 与 SAC 分别属于 on-policy 和 off-policy，这会如何影响样本效率和训练稳定性？这两个算法谁的学习速度更快？谁的性能更好更稳定？

答：PPO 属于 on-policy 方法，每次策略更新主要使用当前策略新采集的数据。虽然 PPO 会对同一批 rollout 进行多轮 minibatch 更新，但旧数据不能像 replay buffer 中的数据一样长期复用，因此样本效率相对较低。其优势在于训练数据与当前策略分布较为接近，同时 clipped surrogate objective 限制了策略单次更新幅度，因此通常具有较好的优化可控性。

SAC 属于 off-policy 方法，通过 replay buffer 长期保存并重复利用过去采集的 transition，因此一次环境交互可以支持多次后续参数更新，通常具有更高的样本效率。不过，off-policy critic 同时涉及 replay buffer、bootstrapping 与函数逼近，旧数据分布与当前策略也可能存在偏移，因此 Q function 的估计误差可能影响 actor 更新。SAC 通过 twin Q、target network 以及 maximum-entropy objective 等设计提高训练稳定性。

从本实验结果来看，以 interaction step 衡量时，SAC 的学习速度总体明显快于 PPO。例如在 halfCheetah 和 hopper 中，SAC 均以更少的环境交互达到较高的 evaluation reward，体现出 off-policy 方法较高的样本效率

2. 从策略随机性看，PPO 的 entropy bonus 与 SAC 的 maximum entropy objective 有什么异同？

答：算法层面，PPO 是在 total objective 中融合了带系数的 entropy bonus，使得神经网络参数更新时会由于 entropy bonus 的存在鼓励未来策略随机性高的行为，本身 Bellman target 并没有改变形式；而 SAC 是直接 Bellman target 中融入 entropy 项形成 soft Bellman target，再通过在 objective 中引入 temperature 来控制未来策略随机性。

SAC 不是简单地把 entropy 权重设得更大，而是把 entropy 从一个辅助正则项提升成了算法目标和价值函数定义的一部分，相比 PPO 中作为辅助正则项的 entropy bonus，SAC 在算法结构上对 entropy 的依赖更核心。

4 总结与反思

1. 通过实现 PPO，你对 clipped surrogate objective 是否有了新的理解？

答：原来对 clipped surrogate objective 的理解是通过限制 ratio 的范围在 $[1-\epsilon, 1+\epsilon]$ 内控制新旧策略的差异，现在看来这种算法的核心是通过修改 surrogate objective，使得样本某个动作概率已经沿着优化方向产生较大更新后就不再给予继续优化的奖励。

在代码实现中，我还进一步认识到 clipping 本质上是一种稳定性与优化速率之间的权衡。它可以降低同一批 on-policy 数据被多次更新时策略变化过大的风险，使旧数据构造的 surrogate objective 仍具有一定可靠性；但如果 clipping range 过小，也可能使部分原本有益的策略更新过早失去梯度，从而使策略更新过于保守。

2. 通过实现 SAC，你对最大熵强化学习是否有了新的认识？

答：我对最大熵强化学习的理解一开始是在奖励之外额外鼓励探索，后来发现更准确的理解是：entropy 并不是附加在算法中的一个辅助 bonus，而是直接参与了策略价值的定义，形成 soft Bellman backup，并贯穿 policy evaluation 与 policy improvement 两个过程。

在实际实现和 temperature sensitivity 实验中，我还进一步认识到 entropy 本质上体现了 exploration 与 exploitation 之间的权衡。较小的 temperature α 使策略更强调当前 critic 给出的高价值动作，可能提高短期利用效率，但也更容易使策略过早集中；较大的 α 则使策略保持更高随机性，能够增强探索和动作空间覆盖，但也会牺牲一定的早期学习速度，同时不同环境对这一权衡的敏感程度并不完全一致，如 hopper 和 halfcheetah。

实验总结

本实验完成了 PPO 与 SAC 两种深度强化学习算法的核心实现与连续控制实验，并围绕 PPO implementation details 与 SAC fixed temperature 进行了针对性对照。最终结论应以实际训练曲线和内部指标为依据，并结合算法机制解释不同设计对优化稳定性、样本利用和策略随机性的影响。